1. Allgemeine Grundkenntnisse (I1-ID: mq4g9vq04ih0)

1. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Eine Spezifikation ist eine präzise Beschreibung der erlaubten Ausgaben.
   - (B) Algorithmen beschreiben Handlungsvorschriften (z.B. zur Berechnung eines Werts).
   - (C) Programmentwicklung heißt immer wieder testen, dann compilieren, dann editieren.
   - (D) Compiler laden Maschinencode zusammen mit Klassenbibliotheken in das Laufzeitsystem.
2. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Der Befehlszähler im von Neumann-Prozessormodell speichert arithmetische Befehle.
   - (B) Interaktive Programme können auf Daten von der Standard-Eingabe reagieren.
   - (C) Die Pipe der bash-Shell ist der Stack der noch nicht verarbeiteten Eingaben.
   - (D) Speicherstellen ist nicht anzusehen, ob sie Zahlen oder Befehle speichern.
3. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Durch Ausgabeumleitung kann man erzeugte Daten in einer Datei abspeichern.
   - (B) Decodieren bedeutet, eine Nachricht als Information zu interpretieren bzw. zu verstehen.
   - (C) Durch N Hexadezimalziffern werden 4mal soviel Informationen dargestellt wie mit N Binärziffern.
   - (D) Der ASCII-Standard wurde zur Lösung des Jahr-2000-Problems durch Unicode ersetzt.
4. Geben Sie die richtige Antwortkombination an (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Strukturierte Daten sind zerlegbar in kleinere Bestandteile.
   - (B) Gleitkomma-Arithmetik ist schneller als Ganzzahl-Arithmetik.
   - (C) Die JVM interpretiert Bytecodebefehle ohne ihre Syntax zu prüfen.
   - (D) Imperative Programmierung heißt, Daten durch Befehle zu ersetzen.

Loading Web-Font TeX/Main/Regular

2. Einige Java-Grundkenntnisse (I1-ID: ia4a8tq09ih0)

1. Von klein nach groß ordnen (0.25P für jede korrekte Angabe)

   Was kann ineinander enthalten bzw. Teil voneinander sein?
   Ordnen Sie von OBEN nach UNTEN die Begriffe vom "kleinsten" bis zum "größten", d.h. **wenn X Teil von Y sein kann, so muss X *weiter oben* stehen als Y**:

   `Anweisung; Ausdruck; Datei; Java-Archiv; Klasse; Klassenpfad; Literal; Methode`

2. Zahlen summieren (0.5P für jede korrekte Angabe)

   Füllen Sie die Lücken, so dass das Programm mit Zahlen A und B als Kommandozeilenparametern aufgerufen Folgendes leistet:
   Es wird die Summe der *ganzen* Zahlen ausgeben, die zwischen A und B (einschließlich) liegen.

```
class Intervallsumme {
    public static void main(String[] args) {
        ;

        int ;
        while ( ) {

        }
        ;
    }
}
```

Loading Web-Font TeX/Main/Regular

3. Grundlegende Datentypen (I1-ID: lw64f0a15ih0)

1. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) Das Tabulatorzeichen ist ein Steuerzeichen.
   - (B) Unicode ist eine Teilmenge von iso8859-1.
   - (C) `"C" + '1'` hat den gleichen Wert wie `'C'+1`.
   - (D) `"C" + 1` ist *kein* erlaubter Java-Ausdruck.
2. Wählen Sie die richtige Antwortkombination aus (1P, bei einer Abweichung 0.5P, sonst 0P):
   - (A) `int` ist ein Ganzzahldatentyp, `char` nicht.
   - (B) `double x = Integer.MAX_VALUE + 1;` ist eine erlaubte Anweisung.
   - (C) Der Wert von `(7 % (5 - 2) == 0) || (7 % (5 + 2) == 0)` ist true.
   - (D) Der Wert von (`1000 + 11/12 == 1000 - 11/12)` ist `false`.
3. Geben Sie die zutreffenden Antworten an (entweder Ausgabewert angeben oder Fehlerart (z.B. Syntaxfehler))(0.5P für jede richtige Angabe):
   - (A) `String s = "Hallo", System.out.println(s.length);`

     Ausgabe?
   - (B) `System.out.println(10e-1f);`

     Ausgabe?
   - (C) `2 * (short) 0xC0CA C01A`

     Typ des Ausdrucks?
   - (D) `double y = Double.MIN_VALUE; System.out.println(y/2);`

     Ausgabe?
   - (E) `int x = Integer.MAX_VALUE, y = Integer.MAX_VALUE--, z = x-y;`

     Wert von `z`?
   - (F) `System.out.println( (double) (1/2));`

     Ausgabe?
4. Wählen Sie die richtige Antwortkombination aus (2P, bei einer Abweichung 1P, sonst 0P):
   - (A) `int[] z = {1}; System.out.println(z.length());` produziert die Ausgabe 1.
   - (B) `int[] z = {1}; System.out.println(z[1]); produziert nicht die Ausgabe 1.`
   - (C) Durch `int[][][][] x = new char[32][32][32][16];` wird 1Mb Stack-Speicher reserviert.
   - (D) Durch `int[] a = {1, 2, 3}, b = new int[3];` werden zwei neue Felder angelegt.

Loading Web-Font TeX/Main/Regular

4. Integer-Arithmetik (I1-ID: zzfd4k81gdg0)

Sei `c` eine `int`-Variable mit dem Wert `Integer.MAX_VALUE` (also 2147483647). Welche Textausgabe erzeugen diese Anweisungen?:

```
System.out.println(2147483646-c); //
System.out.println((-1)*c);       //
System.out.println(c+2);          //
System.out.println(c*2-1);        //
```

Loading Web-Font TeX/Main/Regular

5. Iteration in Feldern (I1-ID: 435dkh80w5h0)

```
int[] p = { 1, 0, 7, 3, 5, 4, 6, 2 };
int[] c = { 0, 0, 0, 0, 0, 0, 0, 0 };
int[] q = { 0, 0, 0, 0, 0, 0, 0, 0 };
for (int i = 0; i < 8; i++)
    c[i] = p[p[i]];
for (int i = 0; i < 8; i++)
    q[p[i]] = i;
```

Tragen Sie die Werte der Feldkomponenten *nach* Ausführung des Java-Fragments ein:

Index	0	1	2	3	4	5	6	7
c	0	1	2	3	4	5	6	7
q								

Loading Web-Font TeX/Main/Regular

6. Felder: Fehler oder …? (I1-ID: xb2c64a1gdg0)

Erzeugt untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-):
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich (-):

```
 1:  class FehlerOderWie {
 2:      public static void main(String[] args) {
 3:          int[] a = {2,3,5,7,11};
 4:          int[] b = new int[2];
 5:          b = a;
 6:          b[4] = -4;
 7:          if (a[4] >= 1) System.out.println(1);
 8:          else System.out.println(0);
 9:      }
10:  }
```

Loading Web-Font TeX/Main/Regular

7. Statische Methoden (I1-ID: 0f1dmv607ih0)

```
public class StatischeMethoden {
    public static void q1(int n) {
        n = n * n;
    }
    public static void q2(int[] f, int i) {
        f[i] = f[i] * f[i];
    }
    public static void main(String[] args) {
        int n = 2;
        int[] f = { n, n, n};
        for (int i = 0; i < 3; i++) {
            q1(n);
            q2(f,n);
        }
        System.out.println(n);
        System.out.println(f[n]);
    }
}
```

Wie lautet die erste Ausgabezeile?

Wie lautet die zweite Ausgabezeile?

Loading Web-Font TeX/Main/Regular

8. Rekursion (I1-ID: 4mqjzge16ih0)

```
public static void strange(int n) {
    if ( n <= 2) return;
    System.out.print("(");
    strange(n-2);
    strange(n-3);
    System.out.print(")");
}
```

Welche Ausgabe bewirkt der Aufruf `strange(7)`?

Tipp: Rekursionsbaum benutzen

Loading Web-Font TeX/Main/Regular

9. Binäre Suche protokollieren (I1-ID: i5k29e615ih0)

Betrachten Sie folgende Implementierung der binären Suche in einem aufsteigend sortierten Feld:

```
public static int binarysearch(int[] f, int q) {
    int lo = 0, hi = f.length - 1;
    while (lo<=hi)
    {
        int m=lo+(hi-lo)/2;
        if (q<f[m]) {
            System.out.print("L");
            hi=m-1;
        }
        else if (q>f[m]) {
            System.out.print("R");
            lo=m+1;
        }
        else {
            System.out.print("!");
            return m;
        }
    }
    System.out.print("-");
    return -1;
}
```

Hier ein Ausschnitt aus dem Unit-Test.

```
int[] g = { 1, 1, 3, 5, 6, 7, 8, 9, 9, 9, 10, 11, 11, 12, 13, 15};
int s = g[4];
System.out.println(binarysearch(g,s));
```

Welche Ausgabe wird erzeugt?

Loading Web-Font TeX/Main/Regular

10. Stack und Queue in Aktion (I1-ID: vh5gi9b1aih0)

1. Auf einem Stack werden *in gemischter Reihenfolge* 10 Push- und 10 Pop-Operationen ausgeführt. Es ist bekannt, dass die Push-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Pop der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?
   - (A) 1 2 3 4 5 6 9 8 7 0
   - (B) 0 4 6 5 3 8 1 7 2 9
   - (C) 1 4 7 9 8 6 5 3 0 2
   - (D) 2 1 4 3 6 5 8 7 9 0

   (1P für jede korrekte Angabe)
2. Auf einer Queue werden *in gemischter Reihenfolge* 10 Enqueue- und 10 Dequeue-Operationen ausgeführt. Es ist bekannt, dass die Enqueue-Operationen die Zahlen 0, 1, 2, …, 9 in dieser Reihenfolge einspeichern und dass bei jedem Dequeue der ausgespeicherte Wert ausgegeben wird. Welche Ausgabelisten sind möglich?
   - (E) 4 6 8 7 5 3 2 9 0 1
   - (F) 2 5 6 7 4 8 9 3 1 0
   - (G) 0 1 2 3 4 5 6 7 8 9
   - (H) 4 3 2 1 0 5 6 7 8 9

   (0.25P für jede korrekte Angabe)

Loading Web-Font TeX/Main/Regular

11. Einen Stack kopieren (I1-ID: kw3g7f70y6h0)

Betrachten Sie folgende API von Stacks zum Speichern von `int`-Werten (alle Methoden/Konstruktoren `public`):

`public class Stack`	
`Stack()`	erzeugt einen neuen Stack
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(int i)`	speichert `i` in diesem Stack
`int pop()`	in diesem Stack zuletzt gespeicherter Wert wird *geliefert* und *entfernt*

Stellen Sie die Zeilen für den Rumpf einer weiteren Instanzmethode

```
public Stack copy() {
  // Rumpf (Ihre Aufgabe)
}
```

in richtiger Reihenfolge zusammen, so dass ihr Aufruf einen Stack mit gleichem Inhalt wie dieser liefert (d.h. es soll dann möglich sein, Original und Kopie unabhängig zu benutzen: Einspeichern beim Original ändert nicht den Zustand der Kopie usw.).

Idee: (1) alle Werte aus diesem Stack zunächst aus- und in einen neuen Stack `h` einspeichern, dann (2) von dort alle Werte aus- und sowohl in diesen Stack als auch in neuen Stack `c` einspeichern, schließlich Referenz der Kopie liefern.

```
Stack h = new Stack(); Stack c = new Stack();

while (!this.empty()) {

h.push( this.pop()); }

while (!h.empty()) {

int i = h.pop();

this.push(i); c.push(i);}

return c;
```

Loading Web-Font TeX/Main/Regular

12. OOP-Verständnisfragen (I1-ID: c6j5qfl08ih0)

Frage 1

Wodurch wird Kapselung bei der objektorientierten Programmierung erreicht?

Antwortmöglichkeiten

Durch `public/private/protected` Instanzvariablen und `public/private/protected` Methoden.

Frage 2

Welche der Konzepte

(A) Polymorphie, (B) Vererbung,(C) Kapselung, (D) Überladen

kann man anhand folgender Situationsbeschreibung illustrieren?

- Jedes *Tier* kann einen Laut von sich geben (ein Geräusch machen).
- Jeder *Hund* ist ein *Tier* und kann einen Laut von sich geben, nämlich: Wau Wau!
- Jede *Katze* ist ein *Tier* und kann einen Laut von sich geben, nämlich: Miau!

Loading Web-Font TeX/Main/Regular

13. Vermischtes zu Klassentypen (I1-ID: sy9i2nb0bih0)

1. Objekte verstehen:

   ```
   public class Ninja {
       private int honor;
       public Ninja( int s) {
           honor = s;
       }
   }
   ```

   Zwei `Ninja`-Objekte werden instanziiert:

   ```
   Ninja a = new Ninja(20), b = new Ninja(20);
   ```

   Welchen Wert hat nun der Ausdruck `a == b`?

2. Fehler oder …?

   ```
   public class SillyTestClass {
       private int x, y;
       public SillyTestClass(int x, int y) {
           System.out.println(y);
       }
       public SillyTestClass(String string1, String string2) {
           System.out.println(string1 + string2);
       }
       public static void main (String[] args) {
           SillyTestClass app = new SillyTestClass(2, "Tests");
       }
   }
   ```

   Welche Aussage charakterisiert diesen Code am besten?

3. Begriffsverständnis:

   In dem Menü stehen einige Aussagen zu Klassenelementen, darunter *eine einzige falsche.*
   Wählen Sie genau die NICHT ZUTREFFENDE Aussage aus
   `In einer Klasse darf es Methoden geben\, die den gleichen Namen und gleiche Argumenttyplisten haben aber unterschiedliche Ergebnistypen.`

Loading Web-Font TeX/Main/Regular

14. Eine Bibliothek für Permutationen (I1-ID: 8834zrd08ih0)

Permutationen entsprechen `int`-Feldern einer Länge n, die die Werte 0,1,...,n-1 speichern (in beliebiger Reihenfolge).

Definition

1. Das *Quadrat* der Permutation `p`, ist die Permutation `q` mit: `q[i]` ist gleich `p[p[i]]` für alle Indexwerte `i`.
2. Die *Unordnung* der Permutation `p` ist die Anzahl der Indexpaare `(i,j)` mit `i<j` und `p[i]>p[j]`. Zum Beispiel hat (2,0,1) Unordnung 2.

Aufgabe

Schreiben Sie eine Klasse `Permutation` mit den *statischen* Methoden `quadrat(..)` und `unordnung(..)`, die aus jeder Clientklasse aufgerufen werden können und Folgendes leisten:

1. `quadrat(p)` ermittelt und liefert das Quadrat der Permutation `p`
2. `unordnung(p)` ermittelt und liefert die Unordnung von

`p`.

Es ist jeweils gesichert, dass `p` wirklich eine Permutation ist - nicht separat prüfen!

15. Eine einfache Klasse (I1-ID: 3g3j2i70x5h0)

Programmieren Sie eine einfache Klasse `Wein` mit der Angaben zu Weinflaschen abgespeichert werden können.
Jeder Wein zeichnet sich durch seine unveränderliche Sorte aus (z.B. "2001er Goldtropfen"), und durch seinen Preis (z.B. 13,50€).
Die folgenden Operationen sollen durch Methoden/Konstruktoren ermöglicht werden (*wenn nicht anders angegeben*: aufrufbar aus beliebigen Clientklassen):

- Erzeugen einer `Wein`-Instanz unter Angabe der Sorte
- Erzeugen einer `Wein`-Instanz unter Angabe der Sorte und des Preises
- Setzen des Preises einer Instanz (*nur von `Wein`-Methoden aus*),
- Abfragen des Preises einer Instanz
- Abfragen der Anzahl der erzeugten Instanzen und der Summe ihrer Preise.

VERWENDEN SIE FÜR DIE DATEN BITTE DIESE BEZEICHNER:
`sorte`, `preis`, `anzahl`, `gesamt`

Zum Einrücken bitte unbedingt die Tabulatortaste anstelle Leerzeichen benutzen (siehe unten).

Loading Web-Font TeX/Main/Regular